📘 سند رسمی فنی — نسخه ۱.۶.۰

برچسب‌خوان هوشمند دارو و محصولات

LabelVoice

معماری سامانه، ملزومات استاد و جدول انطباق، نحوه کارکرد، معرفی تک‌تک فایل‌ها، همه کتابخانه‌ها و فناوری‌های به‌کاررفته و ارزیابی مدل‌ها — سامانه‌ای دو زیرسیستمی (دسکتاپ + موبایل) که فقط «متن روی بسته» را می‌خواند و پاسخ فارسی ایمن تولید می‌کند.

۲	+۱۲۷	۱۵۰	۱۰۰٪
زیرسیستم هم‌منطق	فایل پروژه	آزمون خودکار	پردازش محلی

👨‍💻 توسعه و طراحی: **حمید جلیلوند و نوید بدری**

■ بخش ۱

نمای کلی محصول

برچسب‌خوان هوشمند (LabelVoice) سامانه‌ای است برای خواندن متن روی بسته‌بندی دارو و محصولات با فناوری OCR و تبدیل آن به پاسخ فارسی مرتب، ساخت‌یافته و قابل پخش صوتی. محصول دو زیرسیستم هم‌منطق دارد: یک نسخه دسکتاپ با پایتون و Streamlit، و یک نسخه موبایل به‌شکل PWA قابل نصب که همه پردازش را روی خود گوشی انجام می‌دهد. مرز این ابزار روشن و تغییرناپذیر است: فقط می‌گوید «روی بسته چه نوشته شده» — نه یک کلمه بیشتر. هیچ توصیه پزشکی، کاربرد دارو، مقدار مصرف یا تشخیصی ارائه نمی‌شود و اطلاعات غایب از اینترنت یا حافظه تکمیل نمی‌گردد.

🧠 رویکرد مبتنی بر شواهد

پاسخ از قواعد و قالب‌های فارسی ساخته می‌شود — بدون هیچ مدل زبانی ابری در حلقه. هر عدد داخل پاسخ باید در شواهد OCR موجود باشد؛ بنابراین «توهم» پاسخ در این معماری ممکن نیست و هر جمله قابل ردیابی به تصویر است.

⛔ خطوط قرمز ایمنی

عبارت «قدرت درج‌شده» جایگزین اجباری «دوز مصرف» است؛ نگهبان ایمنی واژه‌های ممنوع را رد می‌کند، اعداد خارج از شواهد را مسدود می‌کند و ادعای کاربرد، تشخیص یا توصیه پزشکی را همیشه می‌بندد.

■ بخش ۲

ملزومات طرح‌شده توسط استاد (استاد قربانی) و محل تحقق هر الزام

ملزومات این پروژه توسط استاد قربانی در دو دسته تعریف شده است: «۱. تعریف مسئله» و «۲. نمودار معماری (تصویر بسته‌بندی)». در این بخش، هر دو دسته به‌صورت عیناً نقل می‌شود و برای هر الزام، نحوه تحقق در سامانه، فایل(های) دقیق پیاده‌سازی و آزمون ردیابی‌پذیر آن مشخص شده است؛ به این ترتیب انطباق پیاده‌سازی با ملزومات استاد، مستند و قابل بررسی مستقیم است. همین نگاشت در سند HTML (بخش «ملزومات استاد»)، در رابط دسکتاپ (بخش 📌 در Streamlit) و در رابط موبایل (بخش 📌) نیز با همان ساختار آمده است.

◆ دسته ۱ — تعریف مسئله (عین ملزومات استاد)

«کاربر دوربین را به سمت جعبه دارو یا بسته‌بندی یک محصول می‌گیرد. سامانه نام، اطلاعات چاپ‌شده و سایر فیلدهای قابل خواندن را استخراج می‌کند و آن‌ها را به زبان فارسی می‌خواند. برای مثال، روی بسته دارو نام، قدرت درج‌شده و تعداد قرص‌ها وجود دارد. سامانه باید اطلاعات خوانده‌شده را از توصیه‌های پزشکی جدا نگه دارد.»

◆ جدول انطباق دسته ۱ — هر جمله الزام کجا تحقق یافته است؟

الزام (عین ملزومات)	نحوه تحقق و آزمون ردیابی‌پذیر	فایل(های) پیاده‌سازی (دسکتاپ ⇄ موبایل)	وضعیت
«کاربر دوربین را به سمت جعبه دارو یا بسته‌بندی یک محصول می‌گیرد»	دوربین زنده با انتخاب تیزترین فریم از ۳ فریم + جایگزین عکس با دوربین سیستم + بارگذاری از گالری؛ ۱۶ آزمون E2E مرورگر	`mobile-app/js/camera.js + js/app.js ⇄ app.py (st.camera_input / st.file_uploader)`	✅ تحقق یافته
«سامانه نام، اطلاعات چاپ‌شده و سایر فیلدهای قابل خواندن را استخراج می‌کند»	OCR دوموتوره چند-PSM با ادغام فازی + استخراج ۷ فیلد ساخت‌یافته با regex دو زبانه و گره‌زدن هر مقدار به شواهد OCR؛ tests/test_patterns_fields.py + tests-js/run_tests.js	`label_reader/ocr/* + extraction/* ⇄ mobile-app/js/ocr.js + js/extractor.js`	✅ تحقق یافته
«آن‌ها را به زبان فارسی می‌خواند»	پاسخ فارسی قاعده‌محور + خوانش صوتی فارسی (edge-tts/gTTS با رضایت در دسکتاپ؛ speechSynthesis روی خود دستگاه در موبایل)؛ tests/test_tts_llm.py	`label_reader/response/builder.py + tts/* ⇄ mobile-app/js/responder.js + js/app.js`	✅ تحقق یافته
«روی بسته دارو نام، قدرت درج‌شده و تعداد قرص‌ها وجود دارد»	این سه فیلد هسته مدل داده‌اند (name، strength، count) با الگوهای اختصاصی قدرت (mg/µg/mL) و تعداد؛ ۱۴ نمونه زمینه‌سنجی‌شده در manifest.json هر سه را پوشش می‌دهند	`label_reader/extraction/patterns.py + fields.py ⇄ mobile-app/js/patterns.js`	✅ تحقق یافته
«سامانه باید اطلاعات خوانده‌شده را از توصیه‌های پزشکی جدا نگه دارد»	نگهبان ایمنی: واژه ممنوع «دوز مصرف» (جایگزین اجباری: «قدرت درج‌شده»)، مسدودسازی ادعای کاربرد/تشخیص/مقدار مصرف، رد مقدار صفر، الزام حضور هر عدد در شواهد، تزریق سلب مسئولیت؛ آزمون اختصاصی در test_response_builder.py و test_security.py (متن مخرب «توصیه بده» رد می‌شود)	`label_reader/response/guard.py ⇄ mobile-app/js/guard.js`	✅ تحقق یافته

◆ دسته ۲ — نمودار معماری (تصویر بسته‌بندی) (عین ملزومات استاد)

«OpenCV (تشخیص ناحیه برچسب، اصلاح زاویه و بهبود وضوح) ← OCR (استخراج نام، اعداد و متن بسته‌بندی) ← پردازش متن و کنترل فیلدها (نرمال‌سازی نام، استخراج قدرت و تعداد درج‌شده) ← تولید پاسخ فارسی (منطق استخراج اطلاعات یا LLM برای بازنویسی وفادار به متن) ← TTS + Streamlit (خواندن اطلاعات و نمایش متن استخراج‌شده)»

◆ جدول انطباق دسته ۲ — هر مرحله کجا و چگونه پیاده شده است؟

مرحله ملزومات استاد	پیاده‌سازی واقعی در سامانه	فایل دسکتاپ ⇄ موبایل	وضعیت
ورودی: تصویر بسته‌بندی	دریافت و اعتبارسنجی (magic bytes، سقف حجم، ابعاد) + ارزیابی کیفیت (تاری، تاریکی، انعکاس)	`app.py + label_reader/validation.py + quality.py ⇄ mobile-app/js/app.js + js/camera.js`	✅
OpenCV — تشخیص ناحیه برچسب، اصلاح زاویه و بهبود وضوح	لبه‌یابی سوبل + بزرگ‌ترین جزء همبند (برش خودکار جعبه)، صاف‌کردن چرخش ±۱۵ درجه، مقیاس هوشمند، CLAHE، تیزکردن Unsharp، نسخه دودویی	`label_reader/preprocess.py + quality.py ⇄ mobile-app/js/preprocess.js + js/boxdetect.js`	✅
OCR — استخراج نام، اعداد و متن بسته‌بندی	Tesseract 5 (PSM 3+6) با ادغام فازی EasyOCR + (lazy) اختیاری (≤ ۰٫۸)؛ در موبایل Tesseract.js 6 روی WebAssembly با PSM 11 پراکنده و داده زبان محلی	`label_reader/ocr/tesseract_provider.py + easyocr_provider.py ⇄ mobile-app/js/ocr.js`	✅
پردازش متن و کنترل فیلدها — نرمال‌سازی نام، استخراج قدرت و تعداد درج‌شده	نرمال‌سازی نویسه (ي→ی، ك→ک، ارقام فارسی)، regex دو زبانه، وضعیت پنج‌گانه فیلد، تناقض‌یابی و محافظ مقدار صفر	`label_reader/textnorm.py + extraction/patterns.py + extractor.py ⇄ mobile-app/js/textnorm.js + js/patterns.js + js/extractor.js`	✅
تولید پاسخ فارسی — منطق استخراج یا LLM برای بازنویسی وفادار	مسیر اصلی: موتور قاعده‌محور (بدون توهم — هر عدد باید در شواهد باشد)؛ ماژول LLM اختیاری و پیش‌فرض خاموش پیاده شده و در صورت فعال‌سازی، خروجی دوباره از نگهبان عبور می‌کند — یعنی فقط «بازنویسی وفادار» ممکن است	`label_reader/response/builder.py + guard.py + llm/rewriter.py ⇄ mobile-app/js/responder.js + js/guard.js`	✅
TTS + Streamlit — خواندن اطلاعات و نمایش متن استخراج‌شده	نمایش ساخت‌یافته فیلدها و خواندن همان پاسخ تأییدشده؛ در دسکتاپ با رضایت صریح، در موبایل کاملاً روی خود دستگاه	`label_reader/tts/* + app.py (Streamlit RTL) ⇄ speechSynthesis در mobile-app/js/app.js`	✅

📋 **جمع‌بندی انطباق:** هر ۵ جمله الزام دسته ۱ و هر ۵ مرحله دسته ۲ پیاده‌سازی، آزمون و راستی‌آزمایی شده‌اند. در مرحله پاسخ‌سازی، از میان دو گزینه ملزومات («منطق استخراج اطلاعات» یا «LLM برای بازنویسی وفادار»)، گزینه امن‌تر یعنی منطق قاعده‌محور مسیر اصلی است و LLM وفادار به‌صورت اختیاری و پیش‌فرض خاموش موجود است — با عبور اجباری خروجی از نگهبان در هر دو حالت.

◆ اعلام مرحله‌به‌مرحله پردازش، نمایش خروجی و ارزیابی مرحله‌ای (از نسخه ۱.۶.۰)

بر اساس همین ملزومات، پردازش در این نسخه دیگر «جعبه سیاه» نیست؛ در حین اجرا، سامانه به کاربر اعلام می‌کند که «الان کدام بخش از نیازمندی استاد انجام شد». پیام اعلام هر مرحله دقیقاً به این شکل است: «مرحله n از ۵ انجام شد: … — این بخش از نیازمندی استاد برآورده شد» و پیش از شروع پردازش، «تعریف مسئله» (دسته ۱ ملزومات) نیز به کاربر اعلام می‌شود. برای هر مرحله، خروجی واقعی همان مرحله و معیارهای ارزیابی کیفیت آن با دکمه «خروجی و ارزیابی» قابل مشاهده است و در پایان، گزارش ارزیابی مرحله‌ای (JSON) برای بایگانی قابل دانلود است. این رفتار در هر دو زیرسیستم (دسکتاپ Streamlit و موبایل PWA) پیاده شده و با ۱۶ آزمون pytest جدید و آزمون پاریتی Node پوشش داده شده است.

مرحله (الزام استاد)	خروجی نمایش‌داده‌شده	معیارهای ارزیابی کیفیت همان مرحله	فایل دسکتاپ ⇄ موبایل
۱. OpenCV (تشخیص ناحیه برچسب، اصلاح زاویه و بهبود وضوح)	تصویر بهبودیافته + وضعیت برش/چرخش	تیزی (واریانس لاپلاسین) قبل ← بعد، کنتراست، زاویه اصلاحی، نسخه‌های پردازشی	`label_reader/stages.py` + `pipeline.py (stage_callback)` + `preprocess.py` ⇄ `mobile-app/js/app.js` + `js/boxdetect.js`
۲. OCR (استخراج نام، اعداد و متن)	متن خام سطربه‌سطر + نواحی شناسایی‌شده	تعداد قطعه/سطر، میانگین اطمینان موتور (٪)، مجموع نویسه‌ها، تعداد گذر (جهت/چرخش)	`label_reader/ocr/*` ⇄ `mobile-app/js/ocr.js`
۳. پردازش متن و کنترل فیلدها	جدول فیلدها با وضعیت پنج‌گانه و شواهد	تعداد وضعیت‌ها (استخراج/نامطمئن/غایب/متناقض)، پوشش شواهد (ردیابی‌پذیری ٪)	`extraction/*` + `textnorm.py` ⇄ `mobile-app/js/extractor.js` (`onStage` در `js/pipeline.js`)
۴. تولید پاسخ فارسی (استخراج یا LLM وفادار)	پاسخ فارسی نهایی	عبور از نگهبان ایمنی (بدون «دوز مصرف»/توصیه)، مسیر تولید، طول پاسخ	`response/builder.py` + `guard.py` ⇄ `mobile-app/js/responder.js` + `js/guard.js`
۵. TTS + رابط (خواندن و نمایش)	متن آماده خوانش + پخش‌کننده صوتی	موتور صوتی، آماده‌سازی تلفظ، وضعیت صوت (در دسکتاپ با رضایت کاربر)	`label_reader/tts/*` + `app.py` ⇄ `speechSynthesis` در `mobile-app/js/app.js`

🧪 **آزمون این رفتار:** ترتیب مراحل، پیام‌های اعلام، معیارهای هر مرحله و گزارش JSON در آزمون‌های tests/test_stages.py (۱۶ آزمون) و آزمون پاریتی Node (اعلام مراحل ۳ و ۴ با onStage) راستی‌آزمایی شده است؛ در رابط‌ها: بخش «اعلام مراحل پردازش و ارزیابی مرحله‌ای» در app.py دسکتاپ و خط‌زمان stages-box در نسخه موبایل، همراه مودال راهنمای اختصاصی در هر دو.

■ بخش ۳

معماری سامانه

دو زیرسیستم مستقل با منطق یکسان طراحی شده‌اند تا تجربه و رفتار محصول روی رایانه و گوشی یکسان باشد؛ هر دو دقیقاً همان قواعد نرمال‌سازی، استخراج، تناقض‌یابی و نگهبان ایمنی را پیاده می‌کنند و خروجی JSON هم‌ساختار تولید می‌کنند. این «تطبیق دوقلو» با آزمون‌های موازی (pytest در برابر Node) تضمین می‌شود.

🖥️ **زیرسیستم دسکتاپ — Python**

رابط Streamlit با RTL کامل؛ پیش‌پردازش با OpenCV (CLAHE، حذف نویز، تیزکردن، صاف‌کردن چرخش، سه نسخه خروجی)؛ OCR با Tesseract 5 (PSM 3+6 و ادغام فازی) و EasyOCR اختیاری؛ مدل داده با Pydantic v2 و وضعیت پنج‌گانه فیلد؛ گفتار با edge-tts/gTTS فقط با رضایت صریح؛ ۷۹ آزمون pytest.

📱 **زیرسیستم موبایل / PWA — JavaScript**

HTML/CSS/JS خالص (ES5/IIFE) بدون چارچوب سنگین؛ دوربین زنده با getUserMedia (دوربین پشتی) و انتخاب تیزترین فریم از ۳ فریم؛ تشخیص جعبه دارو با سوبل و مورفولوژی؛ OCR با Tesseract.js 6 روی WebAssembly و داده زبان محلی fas+eng؛ گفتار با Web Speech API روی خود دستگاه؛ ۵۴ آزمون ۱۶ + Node آزمون E2E مرورگر.

🔗 **پل هم‌منطقی:** قواعد نرمال‌سازی نویسه، الگوهای regex، ادغام فازی (شباهت ≥ ۰٫۸)، محافظ مقدار صفر، تشخیص تناقض، نگهبان ایمنی و ساختار JSON — همگی جفتی (Python ⇄ JS) پیاده و با آزمون‌های هم‌راستا تأیید شده‌اند.

■ بخش ۴

چگونه کار می‌کند؟ — خط پردازش ۹ مرحله‌ای

این خط در هر دو زیرسیستم با همان ترتیب اجرا می‌شود؛ تفاوت فقط در پیاده‌سازی (OpenCV ⇄ Canvas) است، نه در قواعد. از نسخه ۱.۶.۰ تکمیل هر مرحله هنگام پردازش به کاربر اعلام و خروجی و ارزیابی کیفیت همان مرحله نمایش داده می‌شود (جدول کامل در بخش ۲ همین سند — «اعلام مرحله‌به‌مرحله»).

۱ **ورود و اعتبارسنجی تصویر** File API · magic bytes

بررسی امضای باینری PNG/JPEG/WebP، سقف حجم ۱۰ مگابایت و حداقل ابعاد — فایل غیرتصویری رد می‌شود تا موتور OCR با ورودی مخرب روبه‌رو نشود.

۲ **ارزیابی کیفیت** واریانس لاپلاسین · هیستوگرام

تاری، تاریکی و انعکاس نور سنجیده می‌شود و هشدارها به‌صورت چیپ رنگی نمایش داده می‌شوند تا کاربر در صورت نیاز عکس بهتری بگیرد.

۳ **پیش‌پردازش** OpenCV · Canvas

مقیاس هوشمند تا ۱۲۰۰ پیکسل کوتاه‌ترین ضلع، CLAHE (دسکتاپ)، تیزکردن Unsharp، صاف‌کردن چرخش ±۱۵ درجه و ساخت نسخه‌های اصلی، تیزشده و دودویی.

۴ **تشخیص و برش جعبه دارو** سوبل + مورفولوژی

ناحیه بسته با لبه‌یابی سوبل، آستانه درصدی، اتساع و بزرگ‌ترین جزء همبند پیدا و از پس‌زمینه برش داده می‌شود؛ نویز OCR به‌شدت کاهش می‌یابد.

۵ **OCR چندنسخه‌ای و چند-PSM** Tesseract 5 · Tesseract.js (WASM LSTM)

هر نسخه تصویر با دو حالت چیدمان (PSM 3 و 6 و در موبایل PSM 11 پراکنده) خوانده می‌شود؛ user_defined_dpi برای دقت تنظیم شده است.

۶ **ادغام فازی خوانش‌ها** شباهت ترتیبی ≥ ۰٫۸

خوانش‌های مشابه ادغام و تکرارها حذف می‌شوند؛ بازده خوانش امتیازدهی می‌شود و در خوانش ضعیف، چرخش‌های ۹۰ درجه به‌طور خودکار آزموده می‌شوند.

۷ **استخراج فیلدها و تناقض‌یابی** Regex دو زبانه · Pydantic

نام، قدرت درج‌شده، شکل دارویی، حجم/وزن، تاریخ، سری و سازنده استخراج می‌شود؛ هر مقدار به شناسه قطعه‌های شاهد گره می‌خورد و خوانش‌های متناقض شفاف اعلام می‌شوند.

۸ **تولید پاسخ فارسی** موتور قاعده‌محور

مقدمه بر اساس نوع بسته (دارو یا محصول) انتخاب و پاسخ از قالب‌های فارسی ساخته می‌شود — بدون هیچ مدل زبانی ابری و بدون تکمیل اطلاعات از حافظه.

۹ **نگهبان ایمنی و خروجی** Guard · TTS · JSON

واژه‌های ممنوع رد، اعداد خارج از شواهد مسدود و سلب مسئولیت تزریق می‌شود؛ سپس نمایش، پخش صوتی و دانلود JSON انجام می‌گیرد.

■ بخش ۵

فایل‌های سامانه — مرجع کامل

تک‌تک فایل‌های مخزن به تفکیک دسته، همراه با نقش هر فایل. پوشه‌های داده حجیم (مدل‌ها، نمونه‌ها، آیکون‌ها) به‌صورت گروهی معرفی شده‌اند.

فایل (هسته پایتون — label_reader/)	نقش
`__init__.py`	متادیتا، نام فارسی محصول (APP_NAME_FA)، محدوده (SCOPE_FA) و نسخه
`config.py`	تنظیمات با pydantic-settings: موتور OCR، آستانه‌های کیفیت، ابعاد مقیاس، سقف حجم
`models.py`	مدل‌های Pydantic v2: OcrPiece، RawLine، Field (وضعیت پنج‌گانه + شواهد)، Metadata
`textnorm.py`	نرمال‌سازی نویسه: ي→ی، ك→ک، ارقام فارسی/عربی به لاتین، یکسان‌سازی فاصله
`validation.py`	اعتبارسنجی: magic bytes، سقف حجم/ابعاد، خطای ImageValidationError با پیام فارسی
`quality.py`	تاری (لاپلاسین)، تاریکی، انعکاس؛ هیوریستیک پس‌زمینه‌آگاه و سطوح شدت
`preprocess.py`	OpenCV: مقیاس هوشمند، CLAHE، حذف نویز، تیزکردن، deskew، سه نسخه خروجی
`pipeline.py`	هماهنگ‌کننده کل خط: اعتبارسنجی تا پاسخ و نگهبان
`ui_helpers.py`	کمک‌های رابط: RTL، جدول فیلدها، رسم کادرهای OCR، برچسب‌های فارسی
`extraction/classify.py`	تشخیص نوع بسته: دارو یا محصول عمومی بر اساس واژگان و الگوها
`extraction/patterns.py`	الگوهای regex دو زبانه: قدرت (mg/µg/mL)، شکل دارویی، تاریخ، سری، سازنده
`extraction/fields.py`	نقشه‌برداری قطعه‌ها به فیلدها + ساخت Field با شواهد و یادداشت فارسی
`extraction/conflicts.py`	تشخیص تناقض میان خوانش‌ها با محافظ مقدار صفر
`ocr/tesseract_provider.py`	Tesseract 5 با pytesseract: دو PSM، ادغام فازی ≤ ۰٫۸، کادر و اطمینان
`ocr/easyocr_provider.py`	EasyOCR fa+en — بارگیری lazy برای تصاویر سخت
`response/builder.py`	قالب‌های فارسی پاسخ بر اساس نوع بسته و فیلدهای موجود
`response/guard.py`	نگهبان ایمنی: واژه‌های ممنوع، نظارت اعداد خارج از شواهد، سلب مسئولیت
`response/tts_prep.py`	آماده‌سازی متن برای تلفظ (اعداد و علائم) بدون تغییر معنا
`tts/edge_provider.py · gtts_provider.py`	edge-tts (صداهای عصبی فارسی) و gTTS با پیام وضعیت فارسی
`llm/rewriter.py`	اسکلت بازنویسی اختیاری با LLM — پیش‌فرض خاموش؛ نگهبان همیشه فعال

فایل (نسخه موبایل — mobile-app/)	نقش
`index.html`	پوسته برنامه: هدر، دکمه‌ها، تنظیمات، نتیجه، اورلی دوربین زنده + دکمه‌های راهنما

فایل (نسخه موبایل — mobile-app/)	نقش
`css/style.css`	استایل موبایل‌محور RTL، تم روشن/تاریک خودکار + استایل مودال‌های راهنما
`manifest.webmanifest · sw.js`	مانیفست PWA و Service Worker cache-first برای اجرای آفلاین کامل
`js/textnorm.js · patterns.js · guard.js`	پورت نرمال‌سازی، الگوها و نگهبان ایمنی (یکسان با نسخه پایتون)
`js/extractor.js · responder.js`	استخراج فیلدها + تشخیص تناقض + قالب‌های پاسخ فارسی و مقدمه نوع محصول
`js/pipeline.js`	ارزیابی کیفیت، امتیاز بازده OCR، ساخت نتیجه، اصلاح کاربر
`js/preprocess.js · boxdetect.js`	پردازش Canvas: مقیاس، تیزکردن، دودویی، deskew، چرخش، برش + تشخیص جعبه
`js/ocr.js`	پوشش Tesseract.js: نسخه‌ها، دو PSM، PSM 11، ادغام فازی، گزارش پیشرفت
`js/camera.js`	دوربین زنده getUserMedia: فلاش، زوم، تعویض دوربین، ۳ فریم و انتخاب تیزترین
`js/help.js`	مودال‌های راهنمای درون‌برنامه‌ای + معرفی فناوری هر بخش + مودال ملزومات استاد (reqs) و ارزیابی (eval)
`js/app.js`	کنترل‌گر رابط: رویدادها، پیش‌نمایش، خط پردازش، نتیجه، JSON، TTS
`js/samples-data.js`	نمونه‌های base64 تعبیه‌شده برای آزمایش آفلاین
`label-reader-singlefile.html`	نسخه تک‌فایلی (حدود ۲۸۰ کیلوبایت) — اجرای با دابل‌کلیک بدون سرور
`documentation.html (جدید ۱.۳.۰)`	مستندات فنی مصور و رسپانسیو (نسخه HTML این سند)
`vendor/ · lang/ · assets/fonts/ · icons/`	Tesseract.js 6 (WASM)، داده زبان fas+eng (integer و best)، فونت وزیرمتن و آیکون‌ها
`START-WINDOWS.bat · server-windows.ps1`	لانچر ویندوز ۱۰۰٪ ASCII + سرور TcpListener بدون پایتون (با حفاظت مسیر)

فایل (اندروید، آزمون، اسکریپت، مستندات)	نقش
`android/MainActivity.java`	WebView با مجوز CAMERA، onPermissionRequest و پشتیبانی دوربین زنده
`android/AndroidManifest.xml · build_apk.sh`	مانیفست و اسکریپت ساخت APK (نیازمند JDK + SDK روی ماشین کاربر)
`tests/ (۸ فایل pytest)`	۷۹ آزمون: نرمال‌سازی، الگوها، پاسخ، امنیت، کیفیت، TTS، یکپارچگی
`tests-js/run_tests.js`	۵۴ آزمون Node موازی با pytest (نگهبان‌ها + جعبه + بازده)

فایل (اندروید، آزمون، اسکریپت، مستندات)	نقش
`scripts/make_sample_images.py`	ساخت ۱۴ برچسب نمونه (فارسی/انگلیسی/مخرب/مبهم) با فونت وزیرمتن
`scripts/evaluate.py`	سنجش دقت فیلد روی مجموعه نمونه (۸۵٫۷٪ با Tesseract)
`scripts/run_demo_cli.py · download_tessdata.py · build_singlefile.py`	دموی خطی، دانلود داده زبان، ساخت نسخه تک‌فایلی
`docs/ARCHITECTURE.md · INSTALL.md · LIMITATIONS.md`	معماری متنی، نصب گام‌به‌گام، محدودیت‌های شناخته‌شده
`docs/evaluation/*.json`	نتایج ثبت‌شده ارزیابی (دقت فیلد/حالت، زمان، آزمون‌ها) و اصول سنجش
`models/tessdata/ · sample_images/ · assets/fonts/`	traineddata دسکتاپ، ۱۴ تصویر نمونه + manifest، فونت وزیرمتن

■ بخش ۶

کتابخانه‌ها، ابزارها و فناوری‌ها

همه اجزایی که سامانه روی آن‌ها ساخته شده، با نقش دقیق و محل استفاده:

Python 3.12 — زبان اصلی دسکتاپ و اسکریپت‌ها

OpenCV — پیش‌پردازش تصویر (CLAHE، نویز، تیزی، deskew)

Tesseract 5 + pytesseract — OCR دسکتاپ (LSTM، دو PSM)

EasyOCR — مدل عمیق اختیاری fa+en

Pydantic v2 — مدل داده و تنظیمات با اعتبارسنجی

Streamlit — رابط وب دسکتاپ RTL + مودال @st.dialog

Pillow — باز/تبدیل تصویر و نمونه‌ساز (raqm برای فارسی)

edge-tts — گفتار عصبی فارسی (فقط با رضایت)

gTTS — جایگزین گفتار

Tesseract.js 6 (WASM) — OCR مرورگر، کاملاً محلی

getUserMedia — دوربین زنده پشتی (استاندارد W3C)

Canvas API — پردازش پیکسلی موبایل

Web Speech API — گفتار روی دستگاه (fa-IR)

PWA + Service Worker — نصب و اجرای آفلاین

فونت وزیرمتن — فونت فارسی متن‌باز، محلی و آفلاین

Android WebView — پوسته APK با مجوز دوربین

pytest — ۷۹ آزمون دسکتاپ

Node.js — ۵۴ آزمون واحد موبایل + E2E مرورگر

pandas + Altair — نمودارهای بخش ارزیابی در رابط دسکتاپ (Streamlit)

scripts/evaluate.py — سنجش تکرارپذیر خط لوله روی داده برچسب‌خورده

■ بخش ۷

حریم خصوصی و خطوط ایمنی

🔒 پردازش صددرصد محلی

در هر دو زیرسیستم، OCR و استخراج روی دستگاه اجرا می‌شود. تنها استثنا، گفتار دسکتاپ است که فقط با رضایت صریح کاربر به سرویس TTS ارسال می‌شود؛ نسخه موبایل حتی آن را هم ندارد و گفتار روی خود گوشی انجام می‌شود.

🛡️ نگهبان پاسخ

پاسخ پیش از نمایش بازبینی می‌شود: واژه‌های ممنوع (از جمله «دوز مصرف») رد می‌شوند، هر عدد باید در شواهد OCR موجود باشد و ادعای کاربرد، تشخیص یا توصیه مسدود است؛ سلب مسئولیت به پاسخ تزریق می‌شود.

🗒️ پاسخ مبتنی بر شواهد

هر فیلد به شناسه قطعه‌های OCR گره خورده است؛ کاربر می‌بیند هر عدد از کدام سطر تصویر آمده و در صورت اشتباه، با حفظ تاریخچه آن را اصلاح می‌کند. «قدرت درج‌شده» همیشه جایگزین «دوز مصرف» است.

🚫 مقاومت در برابر ورودی مخرب

نمونه‌های مخرب آزمون نشان می‌دهند نگهبان، دستورهای درون تصویر (مانند «توصیه پزشکی بده») را اجرا نمی‌کند؛ magic bytes، سقف حجم و ابعاد هم در لایه ورودی قرار دارند.

■ بخش ۸

ارزیابی و آزمون مدل‌ها — نتایج واقعی و اصول سنجش

چون موضوع سامانه دارو است و هر اشتباه می‌تواند آسیب‌زا باشد، ارزیابی بر اساس اصول بنیادین سنجش سامانه‌های حساس انجام شده است: زمینه‌سنجی برچسب‌خورده (۱۴ تصویر نمونه با مقادیر واقعی هر فیلد در manifest.json — بدون هیچ عدد جعلی)، سنجش در سطح فیلد (نه کل متن)، شمردن غیبت و تعارض به‌عنوان خطا، نگهبان‌های ضدخطر با آزمون اختصاصی، شفافیت کامل محدودیت‌ها و تکرارپذیری کامل اعداد با **python scripts/evaluate.py**. نتایج کامل در **docs/evaluation/evaluation-results.json** ثبت است و نمودارهای زیر از همین داده ساخته شده‌اند.

🎯 دقت کل فیلدها — ۸۵٫۷٪

۱۲ از ۱۴ فیلد با Tesseract 5 روی مجموعه نمونه — میانگین زمان پردازش ۱٫۹ ثانیه، بیشینه ۲٫۸ ثانیه. سنجش در سطح فیلد انجام شده و غیبت/تعارض هم خطا شمرده می‌شود.

✅ آزمون‌های خودکار — ۱۵۰ پاس

۹۵ آزمون pytest دسکتاپ (شامل ۱۶ آزمون جدید اعلام مراحل و ارزیابی مرحله‌ای) + ۵۵ آزمون پاریتی Node موبایل + ۱۶ آزمون E2E مرورگر — همه پاس. نگهبان‌های ایمنی آزمون اختصاصی دارند.

دقت استخراج به تفکیک فیلد (٪) — Tesseract 5



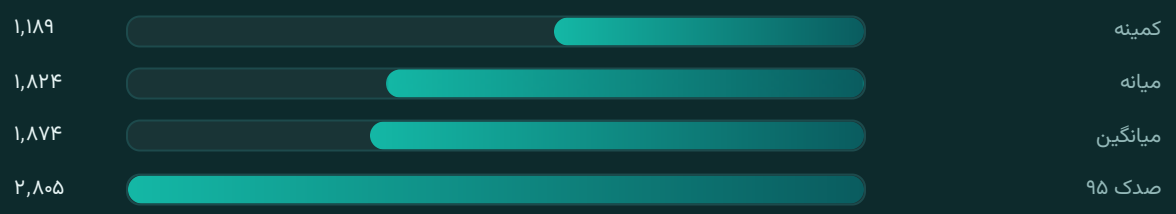
مبنای هر میله: نسبت فیلدهای درست‌خوانده به کل فیلدهای موجود در نمونه‌های برچسب‌خورده (hit/total در evaluation-results.json).

دقت به تفکیک حالت تصویر (٪)

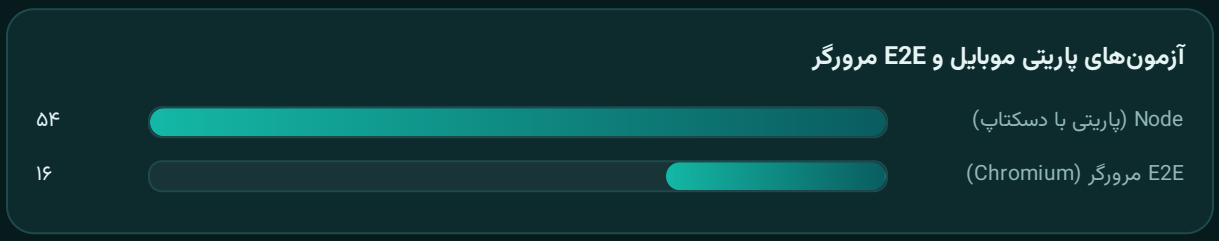


حالت «فارسی ساده» عمداً در مجموعه آزمون نگه داشته شده تا ضعف شناخته‌شده (ارقام فارسی) پنهان نماند — مدیریت آن با وضعیت «نامطمئن» و اصلاح دستی است.

زمان پردازش هر تصویر (میلی‌ثانیه) — Tesseract 5، ۱۴ نمونه







⚠️ **یافته صادقانه:** ارقام فارسی در Tesseract ضعیف خوانده می‌شوند (حالت «فارسی ساده» ٪۰) — به همین دلیل سامانه چنین مقادیری را «نامطمئن» علامت می‌زند و اصلاح دستی ارائه می‌کند؛ هیچ مقدار حدسی به‌عنوان قطعه ارائه نمی‌شود. موتور EasyOCR در محیط این اجرا نصب نبود؛ انتخاب صریح آن با پیام شفاف OcrUnavailableError رد می‌شود (رفتار درست: شکست شفاف، نه سکوت).

🛡️ **مرز ارزیابی با ایمنی:** دقت بالا به معنای مجازشدن توصیه پزشکی نیست. حتی با دقت ٪۱۰۰، این ابزار فقط «چه نوشته شده» را می‌گوید؛ «قدرت درج‌شده» هرگز «دوز مصرف» نامیده نمی‌شود و هر مقدار به شواهد OCR گره خورده است (نگهبان ایمنی: mobile-app/js/guard.js ⇄ label_reader/response/guard.py).

■ بخش ۹

محدودیت‌های شناخته‌شده

🔢 ارقام فارسی در OCR

هر دو موتور در خواندن ارقام فارسی ضعیف‌اند (مثلاً ۵۰۰ گاهی ۰ خوانده می‌شود). این محدودیت مستند شده و با «محافظ مقدار صفر»، وضعیت «نامطمئن» و اصلاح دستی کاربر مدیریت می‌شود؛ پاسخ هرگز عدد مشکوک را قطعی ارائه نمی‌کند.

📷 وابستگی به کیفیت عکس

تاری شدید، انعکاس و زاویه زیاد، دقت را می‌کاهد؛ سامانه این موارد را شفاف اعلام می‌کند و نسخه موبایل با دوربین زنده و انتخاب تیزترین فریم، تأثیر را کاهش داده است.

🧪 تأیید اصالت نیست · LLM پیش‌فرض خاموش

ابزار اصالت دارو را تأیید نمی‌کند و فقط متن ظاهری بسته را می‌خواند. بازنویسی با مدل زبانی به‌صورت اسکلت آمده و پیش‌فرض خاموش است؛ حتی اگر فعال شود، خروجی دوباره از نگهبان عبور می‌کند.

■ بخش ۱۰

نصب و اجرا

🖥️ دسکتاپ

پس از نصب Python و Tesseract (با زبان fas): pip install -r requirements.txt سپس streamlit run app.py.
جزئیات کامل در docs/INSTALL.md.

📱 موبایل (PWA) و اندروید (APK)

آدرس را در مرورگر گوشی باز کنید و «افزودن به صفحه اصلی» را بزنید — اپ نصب و آفلاین اجرا می‌شود. برای اندروید ۷ به بالا، APK آماده (versionCode 9) v1.6.0 از صفحه Releases قابل نصب است؛ برای ساخت دستی: اسکریپت android/build_apk.sh با JDK و Android SDK (راهنما: android/README-ANDROID-fa.md).

■ بخش ۱۱

تاریخچه نسخه‌ها

۷ **اعلام مرحله‌به‌مرحله پردازش + خروجی و ارزیابی هر مرحله** ۲۰۲۶ — v1.6.0

در حین پردازش، سامانه اعلام می‌کند «الان کدام بخش از نیازمندی استاد انجام شد»؛ برای هر مرحله از ۵ مرحله معماری استاد، خروجی واقعی و معیارهای ارزیابی کیفیت با دکمه «خروجی و ارزیابی» نمایش داده می‌شود و گزارش ارزیابی مرحله‌ای (JSON) قابل دانلود است — در هر دو زیرسیستم با ۱۶ آزمون pytest جدید و آزمون پاریتی Node (پیاده‌سازی: label_reader/stages.py + stage_callback در pipeline.py ⇄ خط‌زمان مراحل در js/app.js + onStage در js/pipeline.js)؛ APK بازساخت (versionCode 9)؛ مستند در بخش ۲ همین سند و بخش ۳ سند HTML.

۶ **بخش ملزومات استاد با جدول انطباق** ۲۰۲۵ — v1.5.0

ملزومات استاد قربانی (تعریف مسئله + نمودار معماری) عیناً در هر دو سند (HTML/PDF) و هر دو رابط (دسکتاپ/موبایل) نقل و برای هر الزام، نحوه تحقق، فایل پیاده‌سازی و آزمون ردیابی‌پذیر مشخص شد؛ همین بخش از سند PDF.

۵ **نام توسعه‌دهندگان + بخش ارزیابی مدل‌ها** ۲۰۲۵ — v1.4.0

نام «حمید جلیلوند و نوید بدری» در هدر هر دو سامانه و هر دو سند؛ بخش ارزیابی با اصول سنجش سامانه‌های حساس دارویی و نمودارهای واقعی در هر دو سند و رابط دسکتاپ؛ APK بازساخت (versionCode 7).

۴ **APK بازساخت‌شده از آخرین کد** ۲۰۲۵ — v1.3.1

فایل قابل نصب اندروید (versionCode 6) از کد v1.3.0 ساخته و در Release منتشر شد — شامل دوربین زنده و مودال‌های راهنما که در APK قدیمی v1.0.2 نبود.

۳ **مودال‌های راهنما + مستندات مصور + PDF** ۲۰۲۵ — v1.3.0

راهنمای مودالی در همه بخش‌های برنامه (موبایل و دسکتاپ) با معرفی فناوری هر بخش؛ سند فنی HTML رسپانسیو با انیمیشن و نسخه PDF شکیل.

۲ **دوربین زنده، تشخیص جعبه و کیفیت بالاتر OCR** ۲۰۲۵ — v1.2.0

دکمه دوربین به دوربین زنده واقعی (getUserMedia) تبدیل شد؛ تشخیص خودکار جعبه دارو، ۳ فریم و انتخاب تیزترین، PSM 11، تلاش چرخش ۹۰ درجه و پاسخ غنی‌تر؛ پشتیبانی اندروید.

۱ **رفع اشکال ویندوز و انتشار عمومی** v1.0.1 و v1.0.0

لانچرهای ۱۰۰٪ ASCII برای ویندوز؛ سپس انتشار اولیه با دسکتاپ کامل، PWA آفلاین، ۱۴۹ آزمون خودکار و انتشار روی GitHub Pages و Releases.

فقط متن روی بسته.

همین — و نه یک کلمه بیشتر.

برچسب‌خوان هوشمند (LabelVoice) ابزار خواندن برچسب است؛ نه سامانه تشخیص دارو، تجویز یا توصیه پزشکی. اطلاعات ناخوانا و متناقض به‌صورت شفاف اعلام می‌شوند و همه پردازش‌ها روی دستگاه شما انجام می‌شود.

سامانه: github.com/hjalilvand2004-coder/LabelVoice

نسخه وب: hjalilvand2004-coder.github.io/LabelVoice

سند HTML: …/LabelVoice/documentation.html